Kettenfahrzeug

Semesterprojekt von Lasse, Hossein, Julian

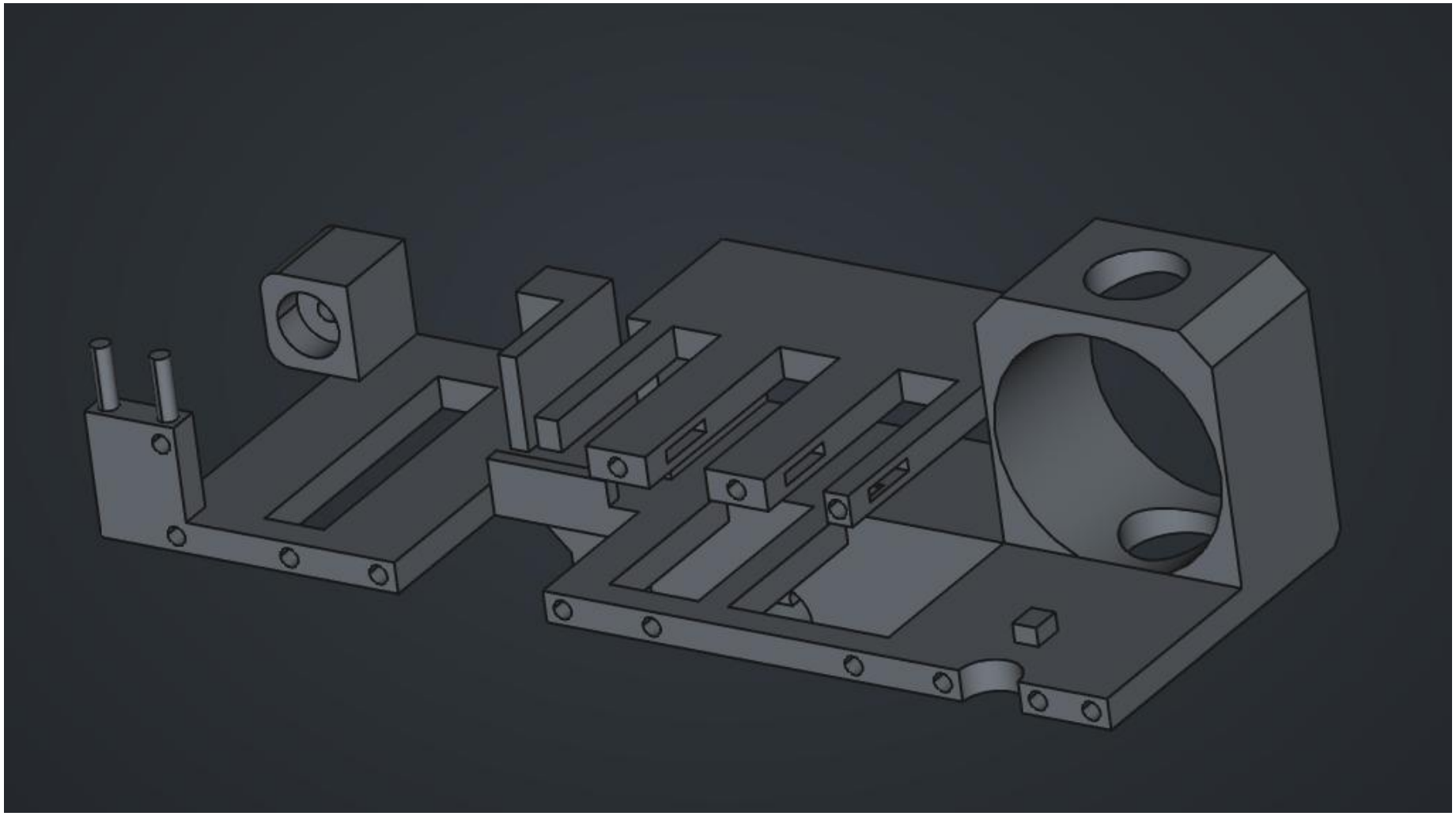
Inhaltsverzeichnis

1. Konstruktion und Planung
2. Elektronik und Verkabelung
3. Programmierung
4. Allgemeines

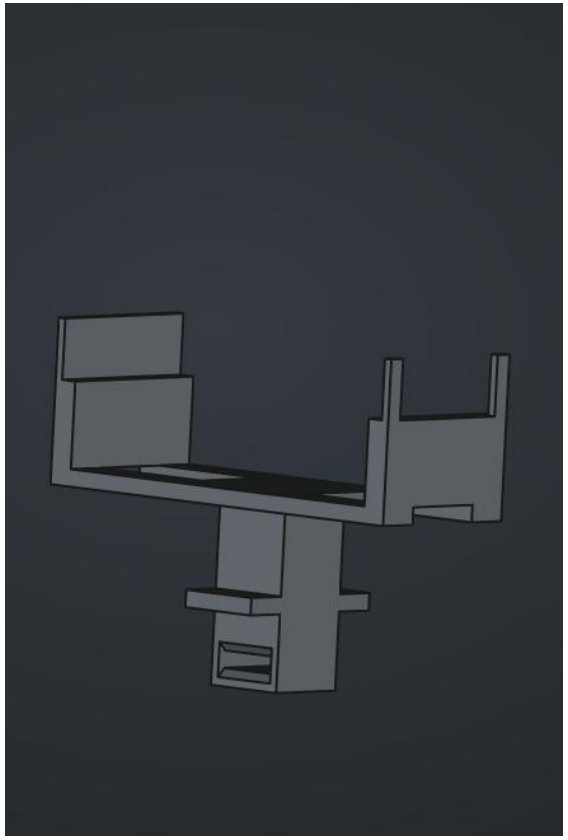
Konstruktion

Lasse

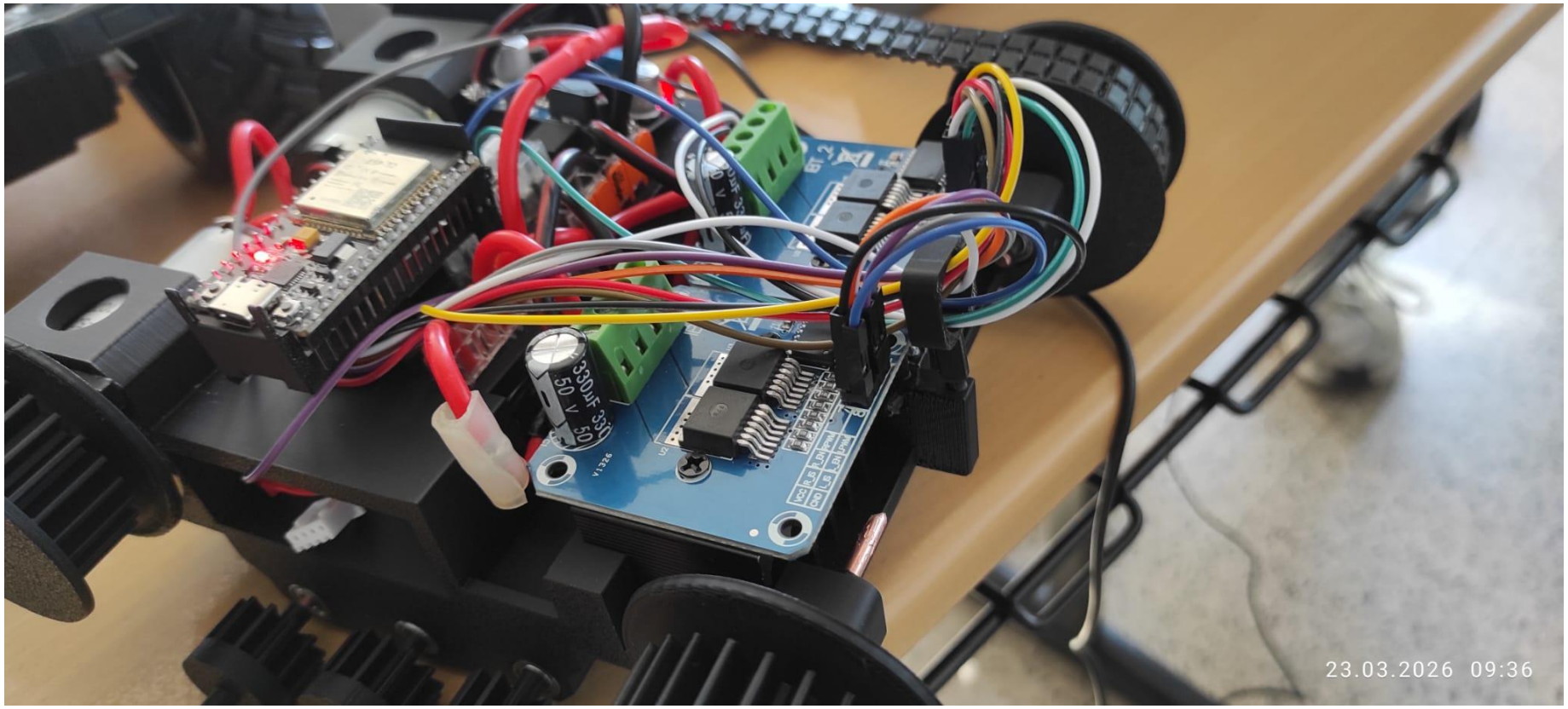
Rahmen



Weitere Teile

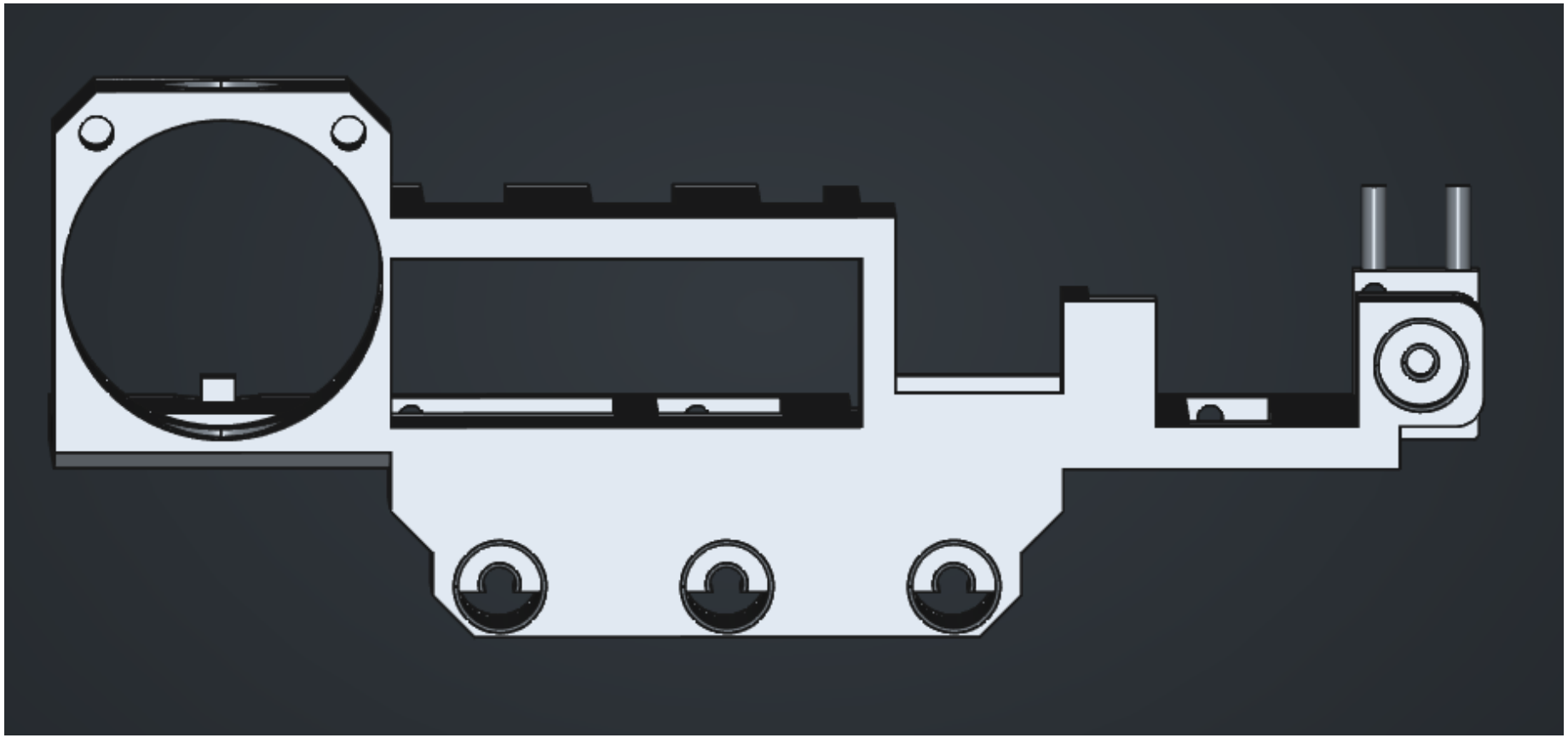


Kabelmanagement

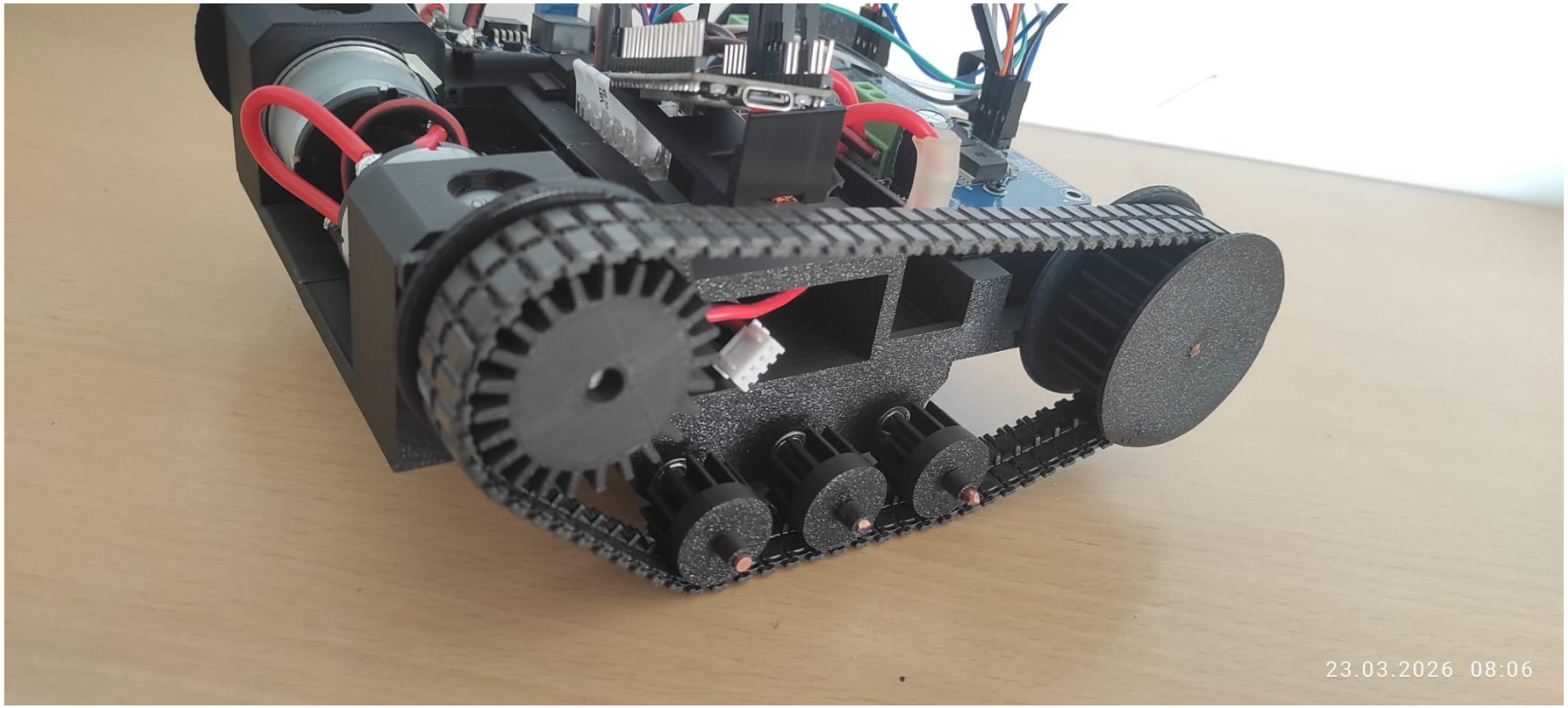


Kabelmanagement

Fahrgestell



Fahrgestell



Zwischenfazit

Positiv

- Doppeltes Fahrgestell
- Modulares System mit vielen externen Teilen
- Flexibilität des Materials ist ein Vorteil

Negativ

- Viele Testdrucke notwendig (Material- und Zeitaufwand)
- Millimeter können ein Modell unbrauchbar machen

Allgemein

- Frühe durchdachte Planung lohnt sich
- 3D-Druck-Ungenauigkeiten antizipieren
- Improvisation ist oft notwendig

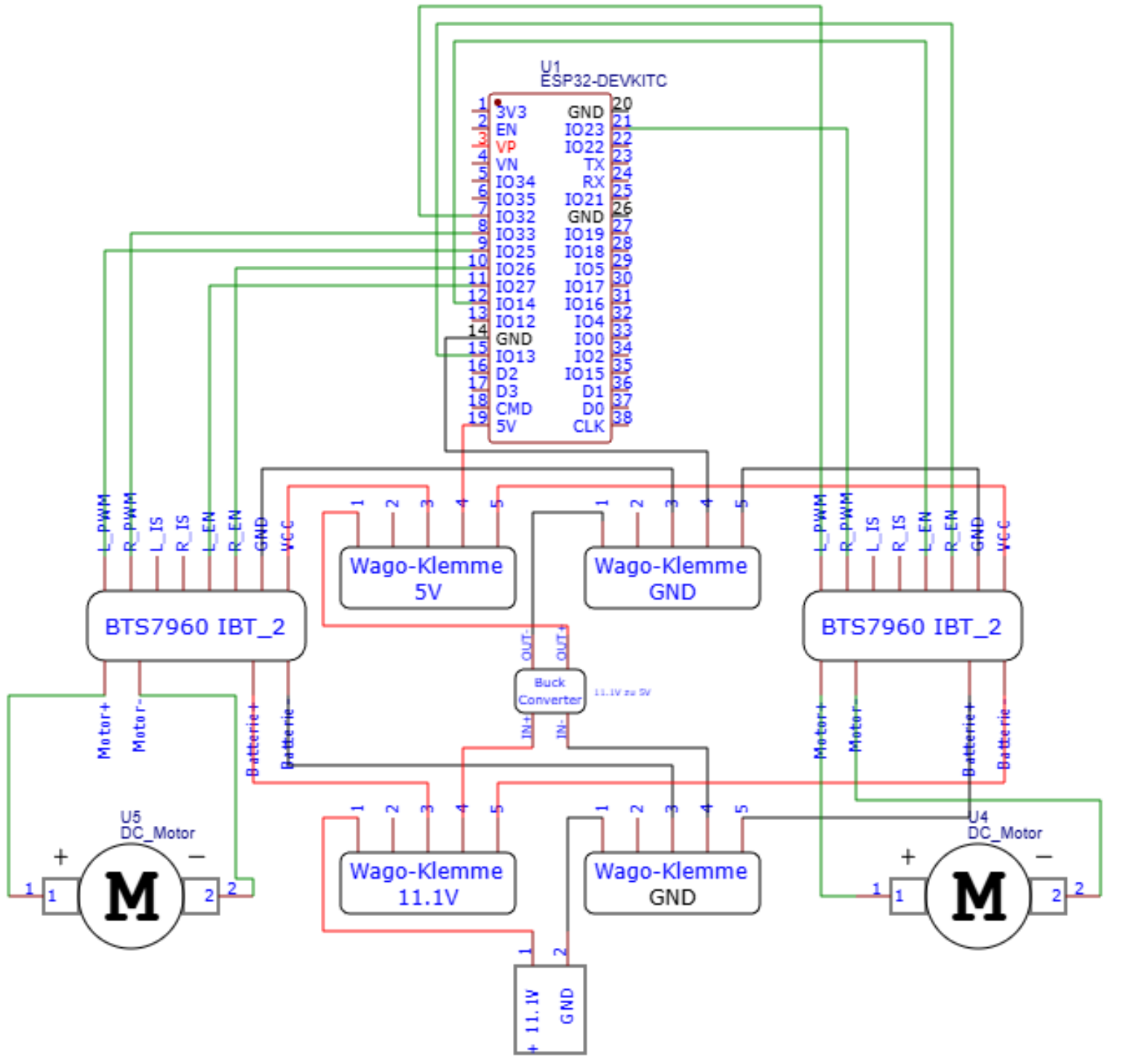
Elektronik

Julian

Elektronik

- 11.1V 3500mAh - Akku
- 2x BTS7960 43A – H-Brücke
- 2x JGB37-520 12V Motor
- LM2596 – Buck-Converter
- ESP32 Dev-Module (38 Pins)

*Alle aufgeführten Bauteile stammen von Aliexpress



https://easyeda.com

Programmierung

Hossein

Programmierung

1. Mit ChatGPT ein Überblick erstellt (nötige Tools usw.)
2. Verbindung zwischen ESP32 und PS5 Controller (Bluepad32)
3. Schnittstelle zwischen Elektronik und Code

1. Schritt

1. Ziel definiert: Was soll der Roboter können?
2. KI gefragt (z.B. ChatGPT/Claude): Weche Komponenten brauche ich? Welche Bibliotheken? Welcher Ansatz?
3. Grobe Architektur skizziert: ESP32 -> Bluetooth -> PS5 Controller -> Motorsteuerung
4. Offene Fragen gesammelt: Was muss ich noch recherchieren?

2. Schritt

1. Youtube & Dokumentation (Bluepad32) als Einstieg
2. Ersten Prototyp erstellt & getestet
3. Stabile BT-Verbindung ohne MAC-Adresse
4. Alle Eingaben live auslesbar (Buttons, Achsen, Trigger)
5. Problem erkannt: Stick Drift / Rauschen bei Nullstellung
6. Lösung: Deadzone + Komplettauslenkung definiert

3. Schritt

1. Verbindungs-Code als Basis genommen & erweitert
2. Achsenwerte auf Motorgeschwindigkeit gemappt
3. Deadzone -> Motoren stehen still
4. Komplettauslenkung -> Maximale Motorleistung
5. Richtungssteuerung implementiert

Code

1. Bibliotheken einbinden
2. Konstanten definieren
3. Controller Array
4. Callback Funktion
5. Setup()
6. Loop()

Bibliotheken

```
#include <Bluepad32.h>
```

Konstanten definieren

```
#define L_EN 27
#define R_EN 26
#define L_PWM 33
#define R_PWM_L 25
#define R_L_EN 14
#define R_R_EN 13
#define R_L_PWM 32
```

```
##define R_R_PWM 23
#define PWM_FREQ 1000
#define PWM_RES 8
#define CH_L_FWD 0
#define CH_L_BWD 1
#define CH_R_FWD 2
#define CH_R_BWD 3
#define DEADZONE 10
```

Controller Array

```
GamepadPtr myGamepads[BP32_MAX_GAMEPADS];
```

Callback Funktion

- Alle 4 Slots im Array durchsuchen
- Ersten leeren Slot (nullptr) suchen
- Controller-Pointer in den freien Slot eintragen
- Falls kein freier Slot → Fehlermeldung im Serial Monitor ausgeben

Setup()

- Serielle Kommunikation starten (Serial Monitor)
- Firmware-Version und Bluetooth-Adresse des ESP32 ausgeben
- Bluetooth-Callbacks bei Bluepad32 registrieren
- Alte Bluetooth-Paarungsschlüssel löschen
- Enable-Pins der beiden BTS7960 auf OUTPUT setzen und dauerhaft HIGH schalten
- LEDC-Kanäle konfigurieren (Frequenz + Auflösung)
- LEDC-Kanäle mit den physischen Pins verbinden

Loop()

- Joystick-Werte des linken und rechten Sticks auslesen
- Deadzone prüfen — wenn Stick in der Mitte → Motor stop
- Richtung bestimmen — negativ = vorwärts, positiv = rückwärts
- Joystick-Wert mit map() auf PWM-Bereich 0–255 skalieren
- PWM-Wert mit constrain() absichern
- PWM-Signal per ledcWrite() auf den Motor schreiben

Allgemeines

Allgemeines

- Github als Kommunikationsplattform
- Ungelöste Probleme
- Arbeitsteilung

Quellen

Code Dokumentation:

https://bluepad32.readthedocs.io/en/latest/plat_arduino/

Zusätzliche Board URL‘s:

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

https://raw.githubusercontent.com/ricardoquesada/esp32-arduino-lib-builder/master/bluepad32_files/package_esp32_bluepad32_index.json